

دليل محرك القواعد ودورة الحياة

كيف تعمل قواعد المحرك (الربط) وقواعد دورة الحياة، وكيف تطبقها في أي جهة أو مؤسسة — مبني على الكود الفعلي.

مثال ترحيل فعلي: **91,504** حساباً · بصفر فشل

مبني على: MappingEngine.cs · LifecycleEngine.cs · SyncEngine.cs

آخر تحديث: 25-07-2026

الفلسفة: Safe Sync — لا حذف ولا تعطيل، فقط إنشاء وتحديث ونقل ومجموعات

♦. العائلتان — لا تخلط بينهما

العائلة	متى تعمل؟	ماذا تقرّر؟	الملف
قواعد المحرك (Mapping)	وقت المزامنة، لكل سجل	قيم سمات AD، وأي OU، وأي مجموعات	MappingEngine.cs
قواعد دورة الحياة (Lifecycle)	في خط الأنابيب الثلاثي	ماذا يحدث للحساب عند تغيير حالته	LifecycleEngine.cs

قاعدة المحرك تجيب: «الموظف رقم 5 — ما يريده وأي OU؟». **قاعدة دورة الحياة تجيب:** «الموظف رقم 5 صارت حالته 7 (متخزج) — انقله للأرشيف وانزع مجموعاته».

١. قواعد المحرك (Mapping Rules)

١.١ عوامل الشرط (Operators)

كل قاعدة لها: ConditionValue · ConditionOperator · ConditionField .

العامل	المعنى	قواعد المحرك (ربط/OU/مجموعات)	قواعد دورة الحياة
==	يساوي (غير حسّاس لحالة الأحرف)	✓	✓
≠	لا يساوي	✓	✓
in	ضمن قائمة مفصولة بفواصل (4,5,6)	✓	✓
not_in	ليس ضمن القائمة	✓	✓
gt / lt	أكبر / أصغر (رقمي)	✓	✗ يُطابق دائماً

فخّ دقيق: gt / lt منقّذان في MappingEngine.EvaluateSimpleCondition (قواعد الربط وOU والمجموعات)، لكن LifecycleEngine.EvaluateCondition تنتهي بـ `≥ true` — فأی عامل غير الأربعة الأولى في قاعدة دورة حياة يُطابق كل هوية بصمت. استخدم gt / lt في قواعد المحرك فقط.

درس STATUS_CODE مقابل STATUSE_CODE: عمود غير موجود ينتج null فلا تطابق أي قاعدة، بصمت — قتل هذا 10 قواعد عبر 111,465 هوية. النظام الآن يضيف اسماً قانونياً STATUS_CODE والبحث غير حسّاس للأحرف.

١.٢ ربط السمات + التحويلات (Transforms)

التحويل	ما يفعله	إدخال	ناتج
(فارغ)	نسخ كما هو	Ali	Ali

التحويل	ما يفعله	إدخال	ناتج
ToUpper / ToLower	حالة الأحرف	ali	ALI
Trim	إزالة الفراغات	" ali "	ali
Format:{0}@corp.com	قالب بقيمة واحدة	a.ali	a.ali@corp.com
Concat:{FIRST} {LAST}	يجمع عدّة أعمدة مصدر	Ali + Saad	Ali Saad
Map:1=Riyadh,2=Jeddah	جدول ترجمة	2	Jeddah
Static:Employee	ثابت يتجاهل المصدر	أي شيء	Employee
Truncate:4	يقصّ الطول	Mohammed	Moha
GetInitials	إن طال (>4) يأخذ أول حرف	Mohammed	M

١.٢- ب توليد اسم المستخدم من الأحرف — Username:

للجهات التي اسم المستخدم فيها مشتقّ من الاسم لا من رقم.

القالب: <[|خيار][|خيار]>:Username

العناصر النائية: {COLUMN} = العمود كاملاً · {COLUMN:N} = أول N حرفاً. النص الحرفي بين العناصر يُحفظ.
الخيارات: lower (افتراضي) · max:N · preserve · upper (افتراضي 20 = حدّ AD).

أمثلة على FIRST_NAME=Mohammed · SECOND_NAME=ali · LAST_NAME=al hareth :

القالب	الناتج
{FIRST_NAME:1}{SECOND_NAME:1}{LAST_NAME}	maalhareth
{FIRST_NAME} . {LAST_NAME}	mohammed.alhareth
{FIRST_NAME:1}{LAST_NAME}	malhareth
{LAST_NAME}{FIRST_NAME:1}	alharethm
{FIRST_NAME}_{EMP_NO}	mohammed_4471
{FIRST_NAME:1}{SECOND_NAME:1}{LAST_NAME} upper	MAALHARETH

سلوك مقصود: كل عمود يُنظّف (حذف الفراغات والتشكيل) قبل أخذ أول N حرفاً — ف {LAST_NAME:3} من «al hareth al» لا al . ثم تُحذف الرموز غير المسموحة، ويُقصّ الناتج إلى 20 حرفاً.

تكرار الاسم: شخصان بنفس الاسم يولّدان نفس القيمة، فيُضاف رقم: `maalhareth2` ← `maalhareth`. قابل للضبط: `UsernameCollisionFormat` (افتراضي) · `UsernameCollisionStart` (2) · `{base}{n}` · `UsernameCollisionMaxAttempts` (20).

١.٢ ج مفتاح المطابقة — `ADMatchAttribute`

المشكلة: إن كان اسم المستخدم مشتقاً من الاسم فهو ليس مفتاحاً ثابتاً. تصحيح إملائي أو تغيير اسم عائلة يولّد اسماً مختلفاً ← النظام لا يجد الحساب ← يُنشئ حساباً مكرّراً في كل مزامنة، بصمت.

الإعداد	المعنى
<code>ADMatchAttribute</code>	سمة AD الحاملة للمفتاح (مثل <code>extensionAttribute2</code>). فارغ = المطابقة بـ <code>sAMAccountName</code> (السلوك الأصلي)
<code>ADMatchSourceColumn</code>	عمود المصدر الذي تُطابق قيمته. فارغ = <code>SourceKeyColumn</code>

كيف يعمل:

- يبحث عن حساب قيمة `extensionAttribute2` فيه = الرقم الوظيفي.
- **وُجد** → يستخدم `sAMAccountName` الموجود كما هو — لا إعادة تسمية، ولا حساب مكرّر.
- **لم يوجد** → هوية جديدة: يولّد الاسم، يفحص توفّره، ثم يبيصم الرقم الوظيفي في السمة عند الإنشاء.

الخطوة الأخيرة هي التي تُغلق الحلقة. لو لم يُبصم الرقم عند الإنشاء، لن تجده المزامنة التالية وستُنشئ حساباً ثانياً — وهكذا كل تشغيلة.

حراسات ضد الفشل الصامت:

- **مطابقة غامضة** (حسابان بنفس الرقم) → يُرفض السجل، لا يُختار أحدهما.
- **فشل قراءة AD** → استثناء صريح، لا يُفسّر كـ «غير موجود».
- **عمود المطابقة فارغ** → يُرفض السجل (لا تمييز بين «جديد» و«تغيّر اسمه»).

١.٢ د اختيار مفتاح المطابقة — أول قرار في أي تركيب جديد

لماذا أولاً: خطأ هذا القرار لا يظهر عند الإعداد، بل بعد أول مزامنة — على شكل حسابات مكرّرة. وتصحيحه بعدها يتطلب دمج حسابات في AD يدوياً.

القاعدة الفاصلة ليست «هل المصدر يحوي اسم المستخدم؟» بل: هل يمكن أن تتغيّر هذه القيمة لنفس الشخص؟

ما يجعل المطابقة بـ `sAMAccountName` آمنة عند الطلاب ليس أن الرقم يأتي من المصدر، بل أن رقم الطالب لا يتغيّر. أما عمود `LOGIN_NAME` في شؤون الموظفين فحقل نصّي يحزّره موظف — وأي تغيير فيه يجعل النظام يبحث عن الاسم الجديد فلا يجده فيُنشئ حساباً مكرّراً.

ما يوقّره المصدر	الربط	لماذا
رقم ثابت فقط (الطلاب)	<code>sAMAccountName</code> — بلا <code>ADMatchAttribute</code>	الاسم مشتقّ من رقم لا يتغيّر

ما يوفّره المصدر	الربط	لماذا
اسم مستخدم مُولّد آلياً ومقفّل	sAMAccountName — شبيه بالطلاب تماماً ✓	القيمة ثابتة فعلياً
اسم مستخدم قابل للتعديل (الشائع)	ADMatchAttribute + الرقم الوظيفي	الاسم للعرض، والرقم للمطابقة
لا رقم ثابت إطلاقاً	❌ غير آمن بنيوياً	لا مفتاح يُطابق به

الحالة الوسطى — المصدر يحوي الاسم والرقم معاً

أفضل تركيبة عملياً، ومدعومة بالإعداد وحده:

ربط الحقول: LOGIN_NAME → sAMAccountName [معرّف] + [مطلوب]
(الاسم جاهز من المصدر - Username: بلا تحويل)

هوية الحساب: ADMatchAttribute = extensionAttribute2
ADMatchSourceColumn = EMP_NO

- حساب جديد → يأخذ اسم المستخدم من شؤون الموظفين كما هو.
- حساب قائم → يُوجَد بالرقم الوظيفي، ويُستخدم اسمه الموجود كما هو.
- تعديل الاسم في HR لاحقاً → لا حساب مكرّر.

مأخذ ١ — لا إعادة تسمية إطلاقاً. UpdateDynamicAsync تتخطى sAMAccountName صراحةً، فتعديل الاسم في HR لموظف قائم لا ينتقل إلى AD. اختيار أمان مقصود: إعادة التسمية تمسّ مسارات الملفات الشخصية وتذاكر Kerberos.

مأخذ ٢ — علّم عمود اسم المستخدم «مطلوب». لو جاء فارغاً تُرجع GetIdentifier قيمة null فيسقط النظام على الرقم اسماً للحساب — فتحصل على حساب اسمه 4471 وسط حسابات بأسماء أشخاص، بلا خطأ.

١.٢-هـ سياسة إنشاء الحسابات — AccountCreationMode

قبل هذا الإعداد كان الإنشاء غير مشروط: أي هوية في المصدر بلا حساب في AD يُنشأ لها حساب تلقائياً.

السياسة	السلوك
Always (افتراضي)	السلوك الأصلي — كل هوية بلا حساب يُنشأ لها
Never	لا يُنشأ أي حساب. القواعد تُطبّق على الحسابات القائمة فقط
Conditional	يُنشأ فقط لمن يحقّق الشرط — مثل «الموظفون النشطون فقط»

فارغ = Always — الجهات المُعدّة قبل وجود هذا الإعداد لا يتغيّر سلوكها.

الرفض عند الشك — ثلاث حالات تُرفض فيها الإنشاء بدل تخمينه:

- شرط ناقص → لا يُنشأ أحد. EvaluateSimpleCondition تُرجع true عند غياب العامل، فالشرط نصف المكتمل كان سيُنشأ الجميع.
- عمود غير موجود في المصدر → يُرفض، والسبب يذكر اسم العمود.

- قيمة سياسة غير معروفة → تُرفض ولا تُخَمَّن.

الملخص في نهاية التشغيل: «لا تُنشئ» تُنتج سطر تخطيط لكل هوية — وهو الشكل الذي يمرّره المشغل بعينه. لذلك يُطبع Provisioning withheld for N identities مع تفصيل الأسباب، حتى لو انتهت التشغيل جزئياً.

١.٣ قواعد الوحدة التنظيمية (OU Rules)

شرط اختياري + قالب ValueMappings + OUPtemplate . أول قاعدة يتحقّق شرطها تفوز.

```
Priority          = 10
ConditionField    = EMP_STATUS , == , 1
OUPtemplate       = OU={DEPT},OU=Employees,{BaseDN}
ValueMappings     = {"DEPT":{"10":"Finance","20":"IT","30":"HR"}}
⇒ DEPT=20        OU=IT,OU=Employees,DC=corp,DC=local
```

عنصر نائب بلا قيمة يُستبدل بـ DEFAULT مع تحذير — وغالباً يُفشل إنشاء الحساب (درس OU=DEFAULT,OU=DEFAULT). بلا قاعدة متطابقة → المرتجع baseDN نفسه.

١.٤ قواعد المجموعات (Group Rules)

افتراضية للجميع: GroupName=All-Staff , IsDefault=true
مشروطة: GroupName=License-IT , DEPT , == , 20

يقبل GroupDN كاملاً أو GroupName . كل القواعد المتطابقة تُجمع (ليس «أول من يفوز»).

٢. قواعد دورة الحياة (Lifecycle Rules)

٢.١ المُطليقات (TriggerType)

TriggerType	يُحمّل؟	الدور
OnImport	نعم	يغيّر الحالة في Metaverse
OnExport	نعم	ينفّذ فعلاً في AD (نقل، مجموعات)
OnSchedule	لا	مذكور، لا يُستعلم عنه — لا يعمل
OnStateChange	لا	مذكور، لا يُستعلم عنه — لا يعمل

٢.٢ الأفعال (ActionType)

ActionType	منفّذ؟	يفعل	ActionValue
SetState	نعم	يغيّر حالة دورة الحياة	Active/Suspended/Deprovisioned

ActionType	منقذ؟	يفعل	ActionValue
MoveOU	نعم	ينقل الحساب	مسار OU، يدعم {BaseDN}
Reactivate	نعم	Active + تفعيل + نقل	OU الوجهة
Deprovision	نعم	Suspended + نزع مجموعات (بلا تعطيل)	OU الأرشيف
Add/RemoveGroups	نعم	مجموعات · فارغ=الكل	أسماء أو {GroupRules}
DisableAD/EnableAD	لا	مذكور، لا case له — بلا أثر	—
SendSMS	لا	مذكور، غير منقذ	—

العنصران النائيان: DN → {BaseDN} الجهة · {GroupRules} في AddGroups → «استنتج المجموعات من قواعد المجموعات» (يحلّ قيد الشرط الواحد — تجنّب قاعدة AddGroups لكل موقع).

الأولوية: الأصغر يُقَيِّم أولاً؛ كل القواعد تُفحص، وآخر `SetState` تطابق هي التي تبقى.
GracePeriodDays: إن $0 <$ ، لا يُنفَّذ الفعل إلا بعد N يوماً على `StateChangedDate`.

٢.٣ خط الأنابيب الثلاثي

- (1) Import → `NeedsRuleEval` حساب البصمة، تعليم، `Metaverse` المصدر إلى
- (2) ApplyRules → تغيير الحالة `OnImport` قواعد
- (3) Export → `AD` تنفَّذ في (في الدفعي `+OnImport`) قواعد `OnExport`

من ينقذ MoveOU؟	القواعد المحمّلة في التصدير
الكاملة/الدفعية	المرحلة 2 تسجّل النية، <code>BulkExport</code> تنفَّذ
الفردية/الدلتا	<code>ApplyLifecycleRules</code> تنفَّذ مباشرة
	<code>OnExport + OnImport</code>
	<code>OnExport</code> فقط

لا توخّذ المسارين بتحميل `OnImport` في التصدير الفردي — يُكرّر كل نقل. يحميه اختبار `FullPipeline_MovesTheAccountExactlyOnce`.

السياسة الافتراضية (بلا قاعدة مطابقة): `Pending → Active`، وما عداها بلا تغيير.

٢.٤ مثال كامل (متخزون — الحالة 7)

- (1) Name=GraduateSuspend , TriggerType=OnImport , Priority=10
ConditionField=STATUS_CODE , == , 7
ActionType=SetState , ActionValue=Deprovisioned
- (2) Name=GraduateMove , TriggerType=OnExport , Priority=20
ConditionField=STATUS_CODE , == , 7
ActionType=MoveOU , ActionValue=OU=Graduates,OU=Archive,{BaseDN}
- (3) Name=GraduateStripGroups , TriggerType=OnExport , Priority=30
ConditionField=STATUS_CODE , == , 7
ActionType=RemoveGroups , ActionValue= (فارغ = كل المجموعات)

لا تستثن Deprovisioned من BulkExport ، وإلا صار النقل للأرشيف مستحيلًا بنويًا. الشرط الصحيح:
LastExportDate == null || StateChangedDate > LastExportDate

٣. جدول القرار — حالة المصدر إلى قواعدها

الرمز	المعنى	OnImport (الحالة)	OnExport (AD)	العدد
1	نشط	— يبقى Active افتراضياً	AddGroups {GroupRules}	—
7	متخزّج	SetState=Deprovisioned	MoveOU=OU=Graduates,{BaseDN} + RemoveGroups	47, 526
10	منسحب	Deprovision (Suspended)	MoveOU=OU=Resigned,{BaseDN}	21, 132
4, 5, 6, 9	حالات نقل	SetState/MoveOU حسب الرمز	نقل OU مطابق	22, 789
2, 3, 11, 12	بلا قاعدة SetState	— تبقى Active (قرار إداري)	تنقلها 1, 7 not_in	57

تحذير الرموز غير المغطاة: كل رمز حالة بلا قاعدة مطابقة يُسجّل باسمه وعدده — راجعه قبل كل مرحلة.

٤. كيف تطبقها في أي مؤسسة (مثال: الموظفون)

الخطوة ١ — الجهة والمصدر

(أعمدته كما هي) HR.V_EMPLOYEES :المصدر
مفتاح التتبع الداخلي ← (رقمي، فريد) EMP_NO :عمود المعرّف
(رقمي) EMP_STATUS :عمود الحالة
(appsettings إلزامي، لا رجوع لـ) خاص بالجهة AD اتصال

مهم: التتبع الداخلي (SyncStates / MetaverseEntries / السجل) يستخدم EMP_NO دائماً — لا اسم المستخدم. اسم المستخدم قيمة في AD فقط.

الخطوة ١-ب — مفتاح المطابقة

ADMatchAttribute = extensionAttribute2
ADMatchSourceColumn = EMP_NO (أو فارغ = عمود المعرف نفسه)

الخطوة ٢ — قواعد المحرك (الربط)

FIRST_NAME → sAMAccountName [معرّف]
Transform=Username:{FIRST_NAME:1}{SECOND_NAME:1}{LAST_NAME}
← Mohammed ali al hareth ينتج maalhareth

EMP_NO → extensionAttribute2 Transform=(فارغ) ← اختياري (يُبصم تلقائياً)
EMP_LOGIN → mail Transform=Format:{0}@corp.com
FIRST_NAME → displayName Transform=Concat:{FIRST_NAME} {SECOND_NAME} {LAST_NAME}
DEPT → department Transform=Map:10=Finance,20=IT,30=HR

OU: Priority=10 , EMP_STATUS==1
OUTemplate=OU={DEPT},OU=Employees,{BaseDN}
ValueMappings={"DEPT":{"10":"Finance","20":"IT","30":"HR"}}

مجموعات: All-Staff (IsDefault) · License-IT (DEPT==20)

لاحظ: Username: يقرأ الأعمدة التي يسميها القالب، فعمود المصدر في قاعدة المعرف لا يؤثر على الناتج. وإن كانت كل أعمدة القالب فارغة فلا يُولد اسم ويُرفض السجل — بدل أن يُستخدم الرقم الوظيفي اسماً بصمت.

الخطوة ٣ — قواعد دورة الحياة

لا : SetState (ينبغي Active) + OnExport/AddGroups={GroupRules}
(10) مستقيل: OnImport/=10/Deprovision/OU=Resigned,OU=Archive,{BaseDN}
(7) متخرج: OnImport/=7/SetState=Deprovisioned
OnExport/=7/MoveOU=OU=Graduates,OU=Archive,{BaseDN}
جامعة: مراجعة (وراقب تحذير الرموز غير المغطاة) OU not_in 1,7,10 →

الخطوة ٤ — التطبيق والتحقق

- جَرِّب على 3-5 حسابات أولاً وتحقق في AD بعينك
- شغّل تجريبياً (dry-run) — تأكد أنه لا يكتب ولا ينقل فعلاً
- طبق مرحلة بمرحلة (رمز حالة واحد كل مرة)، لا دفعة واحدة
- راقب أول دقائق: ارتفاع Failed ← أوقف فوراً
- لا معاملات SQL مفتوحة (BEGIN بلا COMMIT يشلّ التزامنة)

الخطوة ٥ — البصمتان المنفصلتان

ما تُفَرِّغُه	ما تريد إعادة تطبيقه
<code>SyncStates.CurrentHash = ''</code>	سمات AD (قواعد الربط)
<code>MetaverseEntries.NeedsRuleEval = 1</code>	قواعد دورة الحياة
<code>MetaverseEntries.LastExportDate = NULL</code>	إعادة التصدير بعد تغيير حالة

تفريغ البصمات وحده لا يُشغِّل دورة الحياة — شرط SyncEngine يقارن حقول MetaverseEntries لا SyncStates.

تشغيل مرحلة واحدة--

```
UPDATE MetaverseEntries SET NeedsRuleEval = 1, LastExportDate = NULL
WHERE TenantId = 1 AND SourceStatusCode = 7;
```

٥. الخيط الجامع

كل القواعد تتبع نمطاً واحداً: شرط بسيط (Field · Op · Value) ← فعل واحد. القوة في التركيب (قواعد بأولويات)، والخطر في الصمت: اسم عمود خاطئ، أو عامل غير مدعوم في محرّك دون آخر، أو فعل غير منقّذ (SendSMS)، أو مُطْلِق لا يُحمّل (OnSchedule) — كلها تُطابق أو تُتجاهل بلا خطأ.

القاعدة الأولى دائماً: اختبر على عيّنة صغيرة وتحقّق بعينك في AD قبل أي دفعة.